TP1 : Accès aux PORTs par masques

Terminer le TD1 avant de commencer le TP1

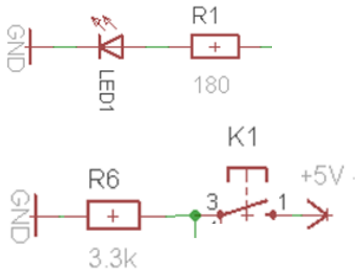
1. Accès aux ports numériques par programmation.

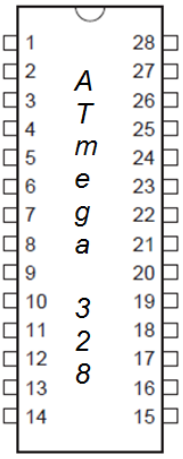
1.1 Exemple

Le but est de trouver un moyen d'accéder aux ports (pattes du composant) par programmation pour connaitre le l'état d'un capteur, ou piloter un actionneur.

Exemple :

Cahier des charges :
Allumer la LED 1 de la carte fille (voir page 5 du TD1) quand le bouton poussoir K1 de la carte fille est appuyé.





Reliez les montages LED1 et K1 aux PIN du microcontrôleur (en vous aidant des schémas page 5 du TD1).

Pour le microcontrôleur, la LED1 est une entrée ou une sortie ?

A quel PORT doit-on accéder pour piloter la LED1 ? En lecture ou écriture ?

Pour le microcontrôleur, le bouton poussoir K1 est une entrée ou une sortie ?

A quel PORT doit-on accéder pour connaitre l'état de K1 ? En lecture ou écriture ?

Le cahier des charges est simple. Mais il faut savoir **lire** l'état logique sur une seule pin et **écrire** le bon état logique sur une seule autre…

1.2 Les registres des ports :

Un registre est un emplacement mémoire du microcontrôleur accessible par programmation. Vous pouvez lire des registres pour savoir ce que fait le microcontrôleur ou écrire dans des registres pour indiquer au microcontrôleur ce que vous voulez faire.

Dans le texte suivant, les parties encadrées sont les parties issues de la documentation technique du microcontrôleur ATmega328.

All registers and bit references in this section are written in general form. A lower case "x" represents the numbering letter for the port, and a lower case "n" represents the bit number. However, when using the register or bit defines in a program, the precise form must be used. For example, PORTB3 for bit no. 3 in Port B, here documented generally as PORTxn. The physical I/O Registers and bit locations are listed in "Register Description" on page 92 (cf ANNEXE 1).

Three registers are allocated for each port :
- the Data Register PORTx,
- the Data Direction Register DDRx,
- the Port Input Pins : PINx.

Each port pin consists of three register bits: DDxn, PORTxn, and PINxn.

The DDxn bit in the DDRx Register selects the direction of this pin. If DDxn is written logic one, Pxn is configured as an output pin. If DDxn is written logic zero, Pxn is configured as an input pin.

If PORTxn is written logic one when the pin is configured as an output pin, the port pin is driven high (one).
If PORTxn is written logic zero when the pin is configured as an output pin, the port pin is driven low (zero).

Independent of the setting of Data Direction bit DDxn, the port pin can be read through the PINxn Register bit.

Lire attentivement ces § et l'annexe 1 et répondez aux questions de la page suivante. Faites vérifier.

thierry.suaton@univ-smb.fr

Combien de registres permettent de gérer les ports B, C et D ? _____
Quels sont leurs noms ? :

Noms des registres qui gèrent le PORT B :

Noms des registres qui gèrent le PORT C :

Noms des registres qui gèrent le PORT D :

Registres de direction :

Quels sont les noms des registres de direction des ports B, C et D ?

Pour le port B :	Pour le port C :	Pour le port D :

Quelle valeur donner à un des bits de ces registres afin de configurer la pin correspondante en entrée ? 0_L / 1_L

Quelle valeur donner à un des bits de ces registres afin de configurer la pin correspondante en sortie ? 0_L / 1_L

Initialiser la direction des ports (ceux non utilisés seront placés en entrée) pour utiliser la carte fille (voir schéma page 5 du TD1). Donner la réponse en binaire et hexadécimal.

DDRB = B	DDRB = 0x
DDRC = B	DDRC = 0x
DDRD = B	DDRD = 0x

Registres d'accès en écriture :

Donner la valeur à PORTD pour allumer les LED4 et LED1 (on placera les bits non utilisés à 0_L). Donner la réponse en binaire et hexadécimal.

PORTD = B	PORTD = 0x

Registres d'accès en lecture :

En écrivant :

```
uint8_t l_valPIND = 0 ;
l_valPIND = PIND ;
```

Quelle valeur aura l_valPIND en supposant que l'on appuie sur les 2 boutons poussoirs de la carte fille (on suppose les bits non utilisés à 0_L) ?

l_valPIND vaudra :	(réponse attendue en binaire et hexadécimal)

On retiendra :

DDRB, DDRC et DDRD sont les registres de direction des PORT.

PORTB, PORTC et PORTD sont les registres d'accès aux PORT en écriture.

PINB, PINC et PIND sont les registres d'accès aux PORT en lecture.

1.3 Accès par masque

thierry.suaton@univ-smb.fr

Dans l'exemple 1.1, pour allumer la LED1 ou l'éteindre il faut accéder uniquement à PD6. Idem, pour lire l'état de K1, il faut lire uniquement PD3. En en utilisant les registres de lecture et écriture (PINB et PORTB), on accède aux 8 ports (de PB0 à PB7) d'un seul coup.
Afin d'accéder à un seul port sur les 8, vous allez procéder par masque.

En utilisant les registres et ce qui a été vu au §1 du TD1 sur les masques, répondez aux questions suivantes et faites vérifier :

Ecrire un port à 0_L :
Donner l'instruction utilisant un masque (faire un #define) et le registre PORTD permettant d'éteindre la LED1. Attention, vous ne devez modifier que l'état de PD6.

Ecrire un port à 1_L :
Donner l'instruction utilisant un masque (faire un #define) et le registre PORTD permettant d'allumer la LED1. Attention, vous ne devez modifier que l'état de PD6.

Lecture d'un port :
Donner les instructions utilisant un masque (faire un #define) et le registre PIND permettant d'affecter une variable de type bool (à déclarer) avec l'état de PD3.

Donner la partie de programme qui permet d'allumer la LED 1 de la carte fille (voir page 5 du TD1) quand le bouton poussoir K1 de la carte fille est appuyé :

2. Environnement de développement intégré (I.D.E.) Arduino :

thierry.suaton@univ-smb.fr

Afin de programmer, nous allons utiliser l'environnement de développement intégré (I.D.E.) Arduino. Il intègre l'éditeur de texte dans lequel on écrit le fichier source (extension .ino), ainsi que les autres fichiers de l'application (fichier de fonctions en extension .cpp et d'entête en extension .h).
Le langage de programmation sera le langage Arduino, très proche du langage C (tout ce que vous avec appris en cours d'info1 est utilisable en Arduino).
Cet environnement permet de compiler votre code et de le téléverser dans le microcontrôleur (via USB).
Il permet enfin de pouvoir faire du débug via la liaison série.

2.1 Création du projet

Exécutez l'IDE Arduino (icone sur bureau). Si le projet (appelé skeatch) n'est pas un nouveau projet, faites Fichier→Nouveau.
Faites Fichier→Enregistrez sous… et enregistrez le projet (nommez le TP1Arduino) dans votre espace (P).

Avec un navigateur Windows, allez dans le dossier TP1Arduino (sur P) que vous venez d'enregistrer. Pour l'instant il ne contient qu'un seul fichier (TP1Arduino.ino).
Dans le dossier TP1Arduino créez un fichier texte (d→Nouveau→Document texte). Renommez le en **TP1.h**
Dans le dossier TP1Arduino créez un fichier texte (d→Nouveau→Document texte). Renommez le en **TP1.cpp**

Dans l'IDE Arduino, ajoutez ces 2 fichiers : Menu Croquis→Ajouter des fichiers…

Vous vous retrouvez donc avec un projet en multifichier :

- Fichier **TP1Arduino.ino** qui contient déjà 2 fonctions loop() et setup(). Ajoutez dans ce fichier l'inclusion du fichier d'entête TP1.h (comme en info1).
- Fichier d'entête **TP1.h** : Ajoutez la directive de compilation `#pragma once`. Ajoutez l'inclusion de la bibliothèque <arduino.h>.
- Fichier source **TP1.cpp** : Ajoutez dans ce fichier l'inclusion du fichier d'entête TP1.h (comme en info1).

Appeler l'enseignant pour vérification.

2.2 Lien avec le matériel

Connecter votre carte microcontroleur au PC. Dans les paramètres systèmes de Windows, récupérer le numéro de port COM attribué à votre carte.
Dans l'IDE Arduino :

- Menu Outils → Programmateur, choisir *AVRISP mkII*
- Menu Outils → Type de carte, choisir *Arduino/Genuino Uno*
- Menu Outils → Port, choisir le bon COM

2.3 Compilation :

Compilez le projet en utilisant l'icône *Vérifier* 

. Le compilateur vérifie les erreurs de syntaxe, effectue le lien entre tous les fichiers de l'application et génère le fichier binaire que l'on pourra téléverser dans la cible (microcontrôleur).
Le message de ce type doit apparaitre dans la fenêtre de sortie :

```
Compilation terminée.
        Compiling debug version of 'TP1' for 'Arduino/Genuino Uno'
        Program size: 2 640 bytes (used 8% of a 32 256 byte maximum) (15,14 secs)
        Minimum Memory Usage: 304 bytes (15% of a 2048 byte maximum)
```

S'il y a des erreurs, la fenêtre de sortie vous indique les erreurs. Il suffit de corriger ces erreurs (sur l'erreur pour que le logiciel vous montre la ligne supposée de l'erreur).

2.4 Chargement du programme dans la cible (téléversement) :

A la différence des TP du cours d'info1, le programme a pour vocation d'être exécuté sur la cible (le microcontrôleur Atmel 328 de la carte Arduino Uno). Il faut donc téléverser le programme (en fait le fichier binaire codant votre programme construit par le compilateur) dans le microcontrôleur (via le cordon USB).

Cette opération se fait par l'icône *Téléverser* 

.
Cette opération entraine un reset du microcontrôleur et donc le commencement de votre programme au début.

2.5 Rappel sur les fonctions setup() et loop() : (dans cette partie ne codez rien, lisez simplement)

Dans l'environnement Arduino, vous n'aurez pas accès directement à la fonction main() comme au semestre 1.

thierry.suaton@univ-smb.fr

En fait, la fonction main() se trouve dans un fichier à part dans lequel vous n'irez jamais coder. Tout le code que vous allez ajouter se trouvera dans les fonctions void setup(void) et void loop(void).

Le main() fait appel de ces fonctions de cette façon :

```
void main (void){
      setup() ;
      do{
            loop() ;
      }while(1) ;
}
```

La fonction setup() sert de fonction d'initialisation, exécutée une seule fois en début de programme (à la mise sous tension du microcontrôleur ou lors d'une demande de reset du programme). On placera dans la fonction setup() les initialisations nécessaires au bon fonctionnement de notre application.
La fonction loop () est exécutée en boucle, à l'infinie (while(1)). C'est donc le programme principal qui gère votre application.

2.6 Premier programme de test de la carte Arduino :

Ce programme permet de tester la carte Arduino. En effet, en projet il est bon de savoir tester le matériel. Si ce programme fonctionne, on est alors certain que la carte Arduino est capable d'exécuter un programme (pas de pannes).
Dans le fichier main.cpp, écrire le programme suivant :

```
void setup() {
      Serial.begin(9600);      //Initialisation de la liaison série à une vitesse de 9600 bits par seconde
}

void loop() {
      Serial.println("OK");    //La carte Arduino envoie la chaine de caractère "OK" au PC via la liaison série (USB)
      delay(1000);             //Attente bloquante (temps indiqué en ms)
}
```

Décrivez ce que doit faire votre programme s'il fonctionne normalement :

Compilez (2.3) et téléversez (2.4). Ouvrez le moniteur (icône ) qui va retranscrire les données reçues par le PC.

Chaque fois que vous serez bloqués en projet par un dysfonctionnement non compréhensible, testez ce programme. N'appelez pas votre enseignant tant que vous n'avez pas au moins fait ce test de base.

3. Programmes d'accès au PORT en utilisant la carte fille (voir pages 5 et 6 du TD1):

Dans ce premier TP, pas de fonctions à créer. On placera tout dans le fichier main.cpp. Dans le second TP, on créera les fonctions.

thierry.suaton@univ-smb.fr

3.1 Fonction setup() de la carte fille :
Complétez la fonction setup() qui permet d'utiliser la carte fille : On doit placer dans cette fonction les assignations des registres DDRB, DDRC et DDRD. Les ports non utilisés seront configurés en entrée.

3.2 Les masques :
Dans le fichier main.cp, définir les masques à utiliser pour les 4 LEDs et les 2 boutons poussoirs (#define).

3.3 Gestion des LEDs (dans la fonction loop() du fichier main.cpp) :
Dans la fonction loop() donner les instructions permettant d'allumer la LED 1. Testez.
Dans la fonction loop() donner les instructions permettant d'éteindre la LED 1. Testez.
Faire un programme qui allume les 4 LEDs (4 instructions utilisant 4 masques et les registres PORTx) pendant 1 seconde, puis qui les éteint (4 instructions utilisant 4 masques et les registres PORTx) pendant 1 seconde. Testez.

3.4 Gestion des boutons poussoirs (dans la fonction loop() du fichier main.cpp) :
Ecrire un programme qui récupère l'état du bouton poussoir K1 et qui allume la LED1 que si K1 est appuyé.

Pour les questions suivantes :
Avant de coder demandez-vous si ce fonctionnement est séquentiel ou combinatoire.
Si le fonctionnement est combinatoire, il suffit de créer 2 variables de type bool, de récupérer l'état des 2 interrupteurs dans ces variables et de piloter les LED en fonction de l'état de ces 2 variables.
Si le fonctionnement est séquentiel, ne codez pas vous ne savez pas faire….

3.5 1er programme de test : Ecrire la fonction loop() en respectant le cahier des charges suivant :
Lorsque le bouton poussoir K1 est appuyé, les LED 1 et 2 doivent s'allumer (éteintes dans le cas contraire).
Lorsque le bouton poussoir K2 est appuyé, les LED 3 et 4 doivent s'allumer (éteintes dans le cas contraire).

3.6 2ème programme de test : Ecrire la fonction loop() en respectant le cahier des charges suivant :
Lorsque le bouton poussoir K1 ou K2 est appuyé, les LED 1 et 2 doivent s'allumer (éteintes dans le cas contraire).
Lorsque les boutons poussoirs K1 et K2 sont appuyés, les LED 3 et 4 doivent s'allumer (éteintes sinon).

3.7 3ème programme :
Le cahier des charges est le suivant : Quand on appui sur le bouton poussoir K1, les 4 LEDs doivent s'allumer et rester allumées quand on relâche K1. Un deuxième appui sur K1 éteint les LEDs.
Ce fonctionnement est séquentiel ou combinatoire ?
S'il est combinatoire codez le cahier des charges avec des expressions logiques (come en §3.5 et 3.6).
Sinon ne le codez pas et passez à la suite.

4. Vitesse d'exécution :

4.1 Test avec les MASQUES.
Dans la fonction loop(), placer uniquement les 2 lignes de code qui permettent :
- d'allumer la LED 1
- d'éteindre la LED 1

Que devrait être le résultat ?

thierry.suaton@univ-smb.fr

Tester et justifier le fonctionnement.
Trouver un moyen de visualiser à l'oscilloscope le signal en sortie du microcontrôleur qui commande la LED1 (sans démonter la carte Arduino). Vous utiliserez pour cela la sonde en x10. La masse (fil noir de la sonde) est accessible sur le métal du connecteur DB9.
Zoomez à l'oscilloscope sur l'état bas du signal. Mesurer sa durée.
Que représente la durée de l'état bas ? Combien de cycles machines représente cette durée ; un cycle machine est une période de l'horloge qui cadence le microcontrôleur (voir question 2.3 du TD1).
Pourquoi la durée de l'état haut est-elle plus importante ?

4.2 Test avec la fonction digitalWrite() de Arduino.
Retentez la même expérience en utilisant la fonction de Arduino *void digitalWrite(int8_t x_i8_pin, int8_t x_i8_value) ;* au lieu de vos 2 lignes de code. Voir https://www.arduino.cc/reference/en/language/functions/digital-io/digitalwrite/
Tester. Mesurer la durée d'un état bas.
Conclure.

Devant ce constat, nous n'utiliserons jamais la fonction digitalWrite() (ni digitalRead()).

5. Schéma équivalent d'une sortie

Lorsqu'une sortie du microcontrôleur impose un 1_L (5V) en sortie, la patte du composant se comporte comme un générateur de tension.
Donner le schéma équivalent d'une sortie en tenant compte de sa résistance de sortie R_S.
Donner la méthode permettant de déterminer la résistance de sortie de ce générateur (utilisation d'une boite à décade). On utilisera la sortie PB3 accessible sur les borniers verts de la carte verte dans lequel vous pouvez insérer un fil rigide pour accéder à une platine LABDEC puis à la boite à décade. Pensez à modifier le code du programme afin d'imposer un 1_L sur PB3. Remarque : La masse est aussi accessible sur les borniers verts de la carte verte.
Appeler un enseignant pour vérifier votre méthode.
Procéder à la mesure de R_S.

6. Mode toggle

6.1 Amélioration de la fonction GereLed1()
Rappeler ce que signifie toggle en informatique. Quel opérateur logique permet d'effectuer un toggle sur un bit ?
Donner le schéma bloc, le prototype et la définition de la fonction GereLed1t() suivante :
Cette fonction reçoit le mode voulu pour la LED1 :

- Si ce mode vaut 0, la LED 1 doit s'éteindre
- Si ce mode vaut 1, la LED 1 doit s'allumer
- Si ce mode vaut 2, la LED 1 doit changer d'état (mode toggle).

Faites un programme de test de cette fonction (en mode toggle) qui fait clignoter la LED1 à une fréquence de 1s (utilisation exceptionnelle de la fonction delay(long int x_duree) qui génère une pause (bloquante) de x_duree ms à chaque appel).

6.2 Mise en évidence des problèmes liés à une pause bloquante
On souhaite en plus gérer ce cahier des charges :
Lorsque le bouton poussoir K2 est appuyé, les LED 3 et 4 doivent s'allumer (éteintes dans le cas contraire).
Coder cette amélioration.
Tester. Conclure sur l'utilisation de pause bloquante en informatique industrielle.

ANNEXE 1
ATmega328 : Register Description

thierry.suaton@univ-smb.fr

13.4.2 PORTB – The Port B Data Register

Bit	7	6	5	4	3	2	1	0	
0x05 (0x25)	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0	PORTB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.3 DDRB – The Port B Data Direction Register

Bit	7	6	5	4	3	2	1	0	
0x04 (0x24)	DDB7	DDB6	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0	DDRB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.4 PINB – The Port B Input Pins Address

Bit	7	6	5	4	3	2	1	0	
0x03 (0x23)	PINB7	PINB6	PINB5	PINB4	PINB3	PINB2	PINB1	PINB0	PINB
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	

13.4.5 PORTC – The Port C Data Register

Bit	7	6	5	4	3	2	1	0	
0x08 (0x28)	–	PORTC6	PORTC5	PORTC4	PORTC3	PORTC2	PORTC1	PORTC0	PORTC
Read/Write	R	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.6 DDRC – The Port C Data Direction Register

Bit	7	6	5	4	3	2	1	0	
0x07 (0x27)	–	DDC6	DDC5	DDC4	DDC3	DDC2	DDC1	DDC0	DDRC
Read/Write	R	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.7 PINC – The Port C Input Pins Address

Bit	7	6	5	4	3	2	1	0	
0x06 (0x26)	–	PINC6	PINC5	PINC4	PINC3	PINC2	PINC1	PINC0	PINC
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	0	N/A	N/A	N/A	N/A	N/A	N/A	N/A	

13.4.8 PORTD – The Port D Data Register

Bit	7	6	5	4	3	2	1	0	
0x0B (0x2B)	PORTD7	PORTD6	PORTD5	PORTD4	PORTD3	PORTD2	PORTD1	PORTD0	PORTD
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.9 DDRD – The Port D Data Direction Register

Bit	7	6	5	4	3	2	1	0	
0x0A (0x2A)	DDD7	DDD6	DDD5	DDD4	DDD3	DDD2	DDD1	DDD0	DDRD
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.10 PIND – The Port D Input Pins Address

Bit	7	6	5	4	3	2	1	0	
0x09 (0x29)	PIND7	PIND6	PIND5	PIND4	PIND3	PIND2	PIND1	PIND0	PIND
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	

CORRECTION

thierry.suaton@univ-smb.fr

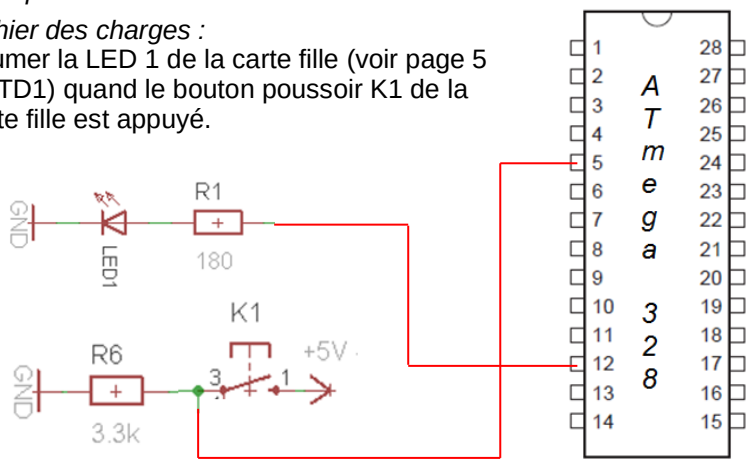
1. Accès aux ports numériques par programmation.

1.1 Exemple

Le but est de trouver un moyen d'accéder aux ports (pattes du composant) par programmation pour connaitre le niveau d'un capteur, ou commander un actionneur.

Exemple :

Cahier des charges :
Allumer la LED 1 de la carte fille (voir page 5 du TD1) quand le bouton poussoir K1 de la carte fille est appuyé.



Reliez les montages LED1 et K1 aux PIN du microcontrôleur (en vous aidant des schémas page 5 du TD1).

Pour le microcontrôleur, la LED1 est une entrée ou une sortie ? ***Sortie***
A quel PORT doit-on accéder pour piloter la LED1 ? En lecture ou écriture ?
PD6 en écriture
Pour le microcontrôleur, le bouton poussoir K1 est une entrée ou une sortie ?
A quel PORT doit-on accéder pour connaitre l'état de K1 ? En lecture ou écriture ?
PD3 en lecture

1.2 Les registres des ports :

Combien de registres permettent de gérer les ports B, C et D ? **9** Quels sont leurs noms ? :

Pour le PORT B : **DDRB, PORTB, PINB.** Pour le PORT C : **DDRC, PORTC, PINC.**
Pour le PORT D : **DDRD, PORTD, PIND.**

Registres de direction :

Quels sont les noms des registres de direction des ports B, C et D ?

Pour le port B : **DDRB** Pour le port C : **DDRC** Pour le port D : **DDRD**

Quelle valeur donner à un des bits de ces registres afin de configurer la pin correspondante en entrée ? **0_L**
Quelle valeur donner à un des bits de ces registres afin de configurer la pin correspondante en sortie ? **1_L**

Initialiser la direction des ports (ceux non utilisés seront placés en entrée) pour utiliser la carte fille (voir schéma page 5 du TD1). Donner la réponse en binaire et hexadécimal.

DDRB = B0000 0000; **DDRB = 0x00;**
DDRC = B0011 0000; **DDRC = 0x30;**
DDRD = B0101 0000; **DDRD = 0x50;**

Registres d'accès en écriture :

Donner la valeur à PORTD pour allumer les LED4 et LED1 (on placera les bits non utilisés à 0_L). Donner la réponse en binaire et hexadécimal.

PORTD = B0101 0000 ; **PORTD = 0x50 ;**

Registres d'accès en lecture :

En écrivant :

```
uint8_t  l_valPIND = 0 ;
l_valPIND = PIND ;
```

Quelle valeur aura l_valPIND en supposant que l'on appuie sur les 2 boutons poussoirs de la carte fille (on suppose les bits non utilisés à 0_L) ?

l_valPIND vaudra : **B0000 1100 ou 0x0C** (réponse attendue en binaire et hexadécimal)

1.3 Accès par masque

thierry.suaton@univ-smb.fr

Dans l'exemple 1.1, pour allumer la LED1 ou l'éteindre il faut accéder uniquement à PD6. Idem, pour lire l'état de K1, il faut lire uniquement PD3. En en utilisant les registres de lecture et écriture (PINB et PORTB), on accède aux 8 ports (de PB0 à PB7) d'un seul coup.
Afin d'accéder à un seul port sur les 8, vous allez procéder par masque.

En utilisant les registres et ce qui a été vu au §1 du TD1 sur les masques, répondez aux questions suivantes et faites vérifier :

Ecrire un port à 0_L :
Donner l'instruction utilisant un masque (faire un #define) et le registre PORTD permettant d'éteindre la LED1. Attention, vous ne devez modifier que l'état de PD6.

```
#define MSQ_LED1  0x40                PORTD = PORTD &~ MSQ_ LED1  ;
```

Ecrire un port à 1_L :
Donner l'instruction utilisant un masque (faire un #define) et le registre PORTD permettant d'allumer la LED1. Attention, vous ne devez modifier que l'état de PD6.

```
#define MSQ_ LED1  0x01               PORTD = PORTD | MSQ_ LED1  ;
```

Lecture d'un port :
Donner les instructions utilisant un masque (faire un #define) et le registre PIND permettant d'affecter une variable de type bool (à déclarer) avec l'état de PD3.

```
#define MSQ_BP1 0x08          bool l_valBp1 ;
                              l_valBp1 = ((PIND & MSQ_ BP1) == MSQ_ BP1);
```

Donner la partie de programme qui permet d'allumer la LED 1 de la carte fille (voir page 5 du TD1) quand le bouton poussoir K1 de la carte fille est appuyé :

```
l_valBp1= ((PIND & MSQ_ BP1) == MSQ_ BP1);
if(l_valBp1== 1){
    PORTD = PORTD | MSQ_MOTEUR ;
}
else {
    PORTD = PORTD &~MSQ_MOTEUR ;
}
```

Ne pas oublier le else. On fera mieux par la suite sans le if else (combinatoire).

2. Environnement de développement intégré (I.D.E.) utilisé :

A la fin de cette partie, vous ne devez pas laisser passer les étudiants à la suite tant qu'ils n'ont pas un projet correct :
Un fichier TP1.h (pour y placer les #define et prototypes fonctions).
Un fichier (dans sources) TP1.cpp (pour y placer les définitions des fonctions).
Un fichier TP1Arduino.ino (pour les fonctions setup() et loop()).

Dans TP1.h :

```
#pragma once
#include <Arduino.h>
```

Dans TP1.cpp :

```
#include "TP1.h"
```

Dans TP1Arduino.ino :

```
#include "TP1.h"

void setup() {

}

void loop() {

}
```

3. Programmes d'accès au PORT en utilisant la carte fille (voir pages 5 et 6 du TD1):

3.1 Fonction setup de la carte fille :

thierry.suaton@univ-smb.fr

Ecrire la fonction setup() qui permettra d'utiliser la carte fille. On doit placer dans cette fonction les instructions permettant de configurer les directions des ports. Les ports non utilisés seront configurés en entrée.

```
void setup(){
      DDRD = 0x50;
      DDRC = 0x30;
      DDRB = 0x00;
}
```

3.2 Les masques :

Dans TP1.h, définir les masques à utiliser pour les 4 LEDs et les 2 boutons poussoirs (#define).

```
#define MASQUE_LED1 0x40
#define MASQUE_LED2 0x20
#define MASQUE_LED3 0x10
#define MASQUE_LED4 0x10
#define MASQUE_K1 0x08
#define MASQUE_K2 0x04
```

3.3 Gestion des LEDs :

Dans la fonction loop() donner les instructions permettant d'allumer la LED 1.

```
PORTD = (PORTD | MASQUE_LED1);
```

Dans la fonction loop() donner les instructions permettant d'éteindre la LED 1. Testez.

```
PORTD = (PORTD & ~MASQUE_LED1);
```

Donner le schéma bloc, le prototype et la définition de la fonction GereLed1() suivante :
Cette fonction reçoit un booléen bMode :

- Si bMode vaut 0, la LED 1 doit s'éteindre
- Si bMode vaut 1, la LED 1 doit s'allumer

Faire de même pour les fonctions GereLed2(),GereLed3() et GereLed4(). Testez.

3.4 Les fonctions de gestion des boutons poussoirs :

Donner le schéma bloc, le prototype et la définition de la fonction GereBpK1() qui retourne l'état (0 ou 1) du bouton poussoir K1. Faire de même pour la fonction GereBpK2().
Placer les définitions des fonctions GereLed1(), GereLed2(), GereLed3(), GereLed4(), GereBpK1() et GereBpK2() dans TP1.cpp.
Dans TP1.h, placer les prototypes des fonctions GereLed1(), GereLed2(), GereLed3(), GereLed4(), GereBpK1() et GereBpK2().

Dans TP1.h :

```
void GereLed1(bool x_mode);
void GereLed2(bool x_mode);
void GereLed3(bool x_mode);
void GereLed4(bool x_mode);
bool GereBpK1(void);
bool GereBpK2(void);
```

Dans TP1.cpp :

```
#include "TP1.h"

void GereLed1(bool x_mode){
      if(x_mode == 0){
            PORTD = (PORTD & ~ MASQUE_LED1);
      }
      else if(x_mode == 1){
            PORTD = (PORTD | MASQUE_LED1);
      }
}


void GereLed2(bool x_mode){
      if(x_mode == 0){
            PORTC = (PORTC & ~ MASQUE_LED2);
```


thierry.suaton@univ-smb.fr

```
        }
        else if(x_mode == 1){
                PORTC = (PORTC | MASQUE_LED2);
        }
}

void GereLed3(bool x_mode){
        if(x_mode == 0){
                PORTC = (PORTC & ~ MASQUE_LED3);
        }
        else if(x_mode == 1){
                PORTC = (PORTC | MASQUE_LED3);
        }
}

void GereLed4(bool x_mode){
        if(x_mode == 0){
                PORTD = (PORTD & ~ MASQUE_LED4);
        }
        else if(x_mode == 1){
                PORTD = (PORTD | MASQUE_LED4);
        }
}

bool GereBpK1(void){
        return ((PIND & MASQUE_K1) == MASQUE_K1);
}

bool GereBpK2(void){
        return ((PIND & MASQUE_K2) == MASQUE_K2);
}
```

3.5 1er programme de test : Ecrire la fonction loop() en respectant le cahier des charges suivant :

Lorsque le bouton poussoir K1 est appuyé, les LED 1 et 2 doivent s'allumer (éteintes dans le cas contraire).
Lorsque le bouton poussoir K2 est appuyé, les LED 3 et 4 doivent s'allumer (éteintes dans le cas contraire).

```
void loop(){
        bool l_testK1 = GereBpK1();
        bool l_testK2 = GereBpK2();
        GereLed1(l_testK1);
        GereLed2(l_testK1);
        GereLed3(l_testK2);
        GereLed4(l_testK2);
}
```

3.6 2ème programme de test : Ecrire la fonction loop() en respectant le cahier des charges suivant :

Lorsque les boutons poussoirs K1 ou K2 sont appuyés, les LED 1 et 2 doivent s'allumer.
Lorsque les boutons poussoirs K1 et K2 sont appuyés, les LED 3 et 4 doivent s'allumer.

```
void loop(){
        bool l_testK1 = GereBpK1();
        bool l_testK2 = GereBpK2();
        GereLed1(l_testK1 || l_testK2);
        GereLed2(l_testK1 || l_testK2);
        GereLed3(l_testK1 && l_testK2);
        GereLed4(l_testK1 && l_testK2);
}
```

3.7 3ème programme : *Programme séquentiel : Donc sera vu plus tard.*

4. Vitesse d'exécution :

4.1 Test avec votre fonction GereLed().

Dans la fonction loop() de TP1Arduino.ino, placer uniquement les 2 lignes de code suivantes :

thierry.suaton@univ-smb.fr

```
GereLed1(0);
GereLed1(1);
```

Que devrait être le résultat ? ***Clignotement LED.***
Zoomez à l'oscilloscope sur l'état bas du signal. Mesurer sa durée. ***125 ns***
Que représente la durée de l'état bas ? ***Durée de l'appel de la fonction GereLed1(1);***
Combien de cycles machines représente cette durée ; un cycle machine est une période de l'horloge qui cadence le microcontrôleur (voir question 1.3 du TD1). ***= 125ns/62.5ns = 2 cycles***
Pourquoi la durée de l'état haut est-elle plus importante ? ***Car il y a le temps d'appel de loop().***

4.2 Test avec la fonction digitalWrite() de Arduino.
Retentez la même expérience en utilisant la fonction de Arduino void digitalWrite(int8_t i8Pin, int8_t i8Value) ; au lieu de GereLed1() : ***digitalWrite(6,0);***
digitalWrite(6,1);
Tester. Mesurer la durée d'un état bas. ***4.62µs. Nbr cycles = 4620ns/62.5ns = 74 cycles !!!***
Conclure. ***Bcp plus lent, problème pour les applications rapides.***

5. Schéma équivalent d'une sortie

Lorsqu'une sortie du microcontrôleur impose un 1_L (5V) en sortie, la patte du composant se comporte comme un générateur de tension.
Donner le schéma équivalent d'une sortie en tenant compte de sa résistance de sortie R_S.
Source de tension 5V en série avec R_S
Donner la méthode :
Méthode de la demi-tension sur PB3. Programme de test :

```
DDRB = 0b00001000;
PORTB = PORTB | 0b00001000;
```

Valeur trouvée : 36Ω

6. Mode toggle

6.1 Amélioration de la fonction GereLed1()

```
void GereLed1t(uint8_t x_mode){
      if(x_mode == 0){
            PORTD = (PORTD & ~ MASQUE_LED1);
      }
      else if(x_mode == 1){
            PORTD = (PORTD | MASQUE_LED1);
      }
      else if(x_mode == 2){
            PORTD = PORTD ^ MASQUE_LED1;  //ou exclusif pour le mode toggle
      }
}
```

Faites un programme de test de cette fonction (en mode toggle) qui fait clignoter la LED1 à une fréquence de 1s (utilisation exceptionnelle de la fonction delay(long int liDuree) qui génère une pause (bloquante) de liDuree ms à chaque appel).

```
GereLed1t(2);
delay(1000);
```

6.2 Mise en évidence des problèmes liés à une pause bloquante
On souhaite en plus gérer ce cahier des charges :
Lorsque le bouton poussoir K2 est appuyé, les LED 3 et 4 doivent s'allumer (éteintes dans le cas contraire).
Coder cette amélioration.

```
GereLed3(l_testK2);
GereLed4(l_testK2);
```

Tester. Conclure sur l'utilisation de pause bloquante en informatique industrielle.
Les appuis peuvent être pris avec jusqu'une seconde de retard…

thierry.suaton@univ-smb.fr